

人工智能赋能的数字集成电路设计作业二

陈天乐

2025 年 11 月 19 日

目录

1 自己修改后的三个 Prompt	2
2 最终设计出来的 RTL 代码和 TestBench	10
3 仿真结果	20

1 自己修改后的三个 Prompt

```

1  ## Matching Submodule
2
3  We are designing a complex digital circuit module. Now there is an implemented RTL IP library.
   Please select the submodules that may be used from it.
4
5  ### Problem Description
6
7  ModelName: [ModelName]
8
9  InputNode: [InputNode]
10
11 OutputNode: [OutputNode]
12
13 Specification: [Specification]
14
15 ### Retrieval RTL Submodules
16
17 | ID      | Name                                | Description
18 | :--- | :----- |
19 | 1      | Multiplexer (MUX)                  | Selects one of several input signals and forwards it to
   a single output line. |
20 | 2      | Decoder                            | Converts a binary code into a one-hot representation,
   activating a single output line. |
21 | 3      | Encoder                            | Performs the inverse function of a decoder; converts a
   one-hot input into a binary code. |
22 | 4      | Full Adder                         | Adds three one-bit inputs (A, B, Carry-in) and produces
   a Sum and Carry-out bit. |
23 | 5      | Ripple-Carry Adder                 | A multi-bit adder constructed by cascading multiple
   Full Adders. Simple but slow. |
24 | 6      | Carry-Lookahead Adder (CLA)        | A high-speed multi-bit adder that calculates carry bits
   in advance to reduce delay. |
25 | 7      | Multiplier                         | A circuit to multiply two binary numbers. Common types
   include Array and Wallace Tree. |
26 | 8      | Comparator                         | Compares two binary numbers and determines their
   relationship (e.g., A > B, A == B). |
27 | 9      | D Flip-Flop                       | A fundamental memory element that stores one bit of
   data on a clock edge. |
28 | 10     | Register                          | A group of D Flip-Flops used to store a multi-bit data
   word. |
29 | 11     | Shift Register                     | A register that can shift its stored data left or right
   on each clock cycle. |
30 | 12     | Counter                           | A sequential circuit that cycles through a
   predetermined sequence of states (e.g., binary count). |
31 | 13     | First-In-First-Out (FIFO)          | A memory buffer that manages data flow between modules,
   ensuring in-order output. |
32 | 14     | Clock Divider                      | A circuit that generates a clock signal with a lower
   frequency from a master clock. |
33 | 15     | Synchronizer                       | A circuit (typically two flip-flops) used to safely
   pass signals between different clock domains. |
34
35
36 Here is an exemplary response, please refer to it, Fill in the answer into the ### Related ID
   segment
37
38 ### Related ID
39
40 ```python

```

```

41 RelatedID = []
42 ...

```

Listing 1: Match_Submodule.md

```

1  ## Design Topmodule
2
3  We are designing a complex digital circuit module. The following is the RTL module that can be
   used directly with the specific problem description.
4
5  ### Problem Description
6
7  ModelName: [ModelName]
8
9  InputNode: [InputNode]
10
11 OutputNode: [OutputNode]
12
13 Specification: [Specification]
14
15 TestBench: [TestSpec]
16
17 ### Retrieval Submodules
18
19
20
21 ### Tips
22
23 Here is some tips that you need to follow:
24
25 1. The module name for RTL is [ModelName], the module name for TestBench is tb_[ModelName].
26 2. You can call these retrieval submodules without add their RTL to the response.
27 3. Please generate a module code to meet the requirements in the Specification, and TestBench
   to meet the requirements in TestBench.
28 4. Please use verilog for RTL and systemverilog for TestBench.
29 5. Fill in the generated RTL into ### RTL, and fill in the generated TestBench into ###
   TestBench.
30 6. We use the -g2005sv specification supported by iverilog for compilation. Please make sure
   that the verilog used can be supported.
31 7. Unpacked structs not supported in iverilog, do not use typedef struct.
32 8. When using the $bitstoreal function, please make sure that the parameter is 64bit.
33 9. You don't need to save waveform through \$.dumpfile and \$.dumvars.
34 10. In Testbench, please verify whether the function is correct. If you pass all the tests,
   please output TestSuccesed, otherwise you will output TestFailed.
35 11. When executing the testbench simulation, please pay attention to the timing of data
   verification and confirm whether the output data has stabilized during data verification
   for the currently designed module.
36
37
38 Your Response Here:
39
40 ### RTL
41
42 ```verilog
43
44 ...
45
46 ### TestBench
47 ```systemverilog
48
49 ...

```

Listing 2: Design_Topmodule.md

```

1 ## Correct Problem
2
3 We are designing a complex digital circuit module. The following is the Problem Description,
  RTL and TestBench code. We use the -g2005sv specification supported by iverilog for
  compilation. Please make sure that the verilog used can be supported. Here is the
  simulation result.
4
5 ### Problem Description
6
7 ModelName: [ModelName]
8
9 InputNode: [InputNode]
10
11 OutputNode: [OutputNode]
12
13 Specification: [Specification]
14
15 TestBench: [TestSpec]
16
17 ### RTL
18
19 ```verilog
20 [RTLCode]
21 ```
22
23 ### TestBench
24
25 ```systemverilog
26 [TestBench_code]
27 ```
28
29 ### Simulation Result
30
31 [SimulationResult]
32
33
34
35 Here is your response. Please judge whether the simulation is passed and whether the function
  is normal through Simulation Result. If all are satisfied, output Success in Result. If
  one item is not satisfied, output Failed in Result.
36
37 ### Result
38
39 ```text
40 Success or Failed
41 ```
42
43 If it is Failed in the previous step. Please give the revised RTL or TestBench and fill in the
  following ### Revised RTL and ### Revised TestBench. If it is Success, this content is
  not required.
44
45 ### Revised RTL
46
47 ```verilog
48
49 ```
50
51 ### Revised TestBench

```

```

52
53 ``systemverilog
54
55 ``
56
57
58 ### Tips
59
60 Here is some tips that you need to follow
61
62 1. If RTL does not need to be modified, please do not generate ### Revised RTL Segment.
    Similarly, if TestBench does not need to be modified, please do not generate ### Revised
    TestBench Segment.
63 2. The module name for RTL is [ModelName], the module name for TestBench is tb_[ModelName].
64 3. You don't need to save waveform through \%dumpfile and \%dumvars.
65 4. Prioritize fixing compilation errors first (e.g., syntax mistakes, undefined modules)
    before addressing functional issues.
66 5. When modifying, keep InputNode and OutputNode of RTL unchanged unless explicitly allowed by
    Specification.
67 6. Prioritize solving the errors shown in the simulation results. Some system functions like "
    break" are not supported.

```

Listing 3: Correct_Problem.md

```

{
  "model_name": "asyn_fifo",
  "input_node": "- wr_clk: Write clock domain clock signal (rising edge effective);\n rd_clk:
  Read clock domain clock signal (rising edge effective);\n rst_n: Active-low asynchronous reset
  signal;\n wr_en: Write enable signal, high effective;\n rd_en: Read enable signal, high
  effective;\n data_in: 16-bit input data to be written to FIFO.",
  "output_node": "- data_out: 16-bit output data read from FIFO;\n full: FIFO full indicator,
  high effective. Only asserted when the write-domain count (wr_count) equals the depth (16), not
  using '>= depth';\n empty: FIFO empty indicator, high effective. Only asserted when the
  read-domain count (rd_count) equals 0, with no early or delayed assertion;\n almost_full: FIFO
  almost full indicator (programmable threshold), high effective;\n almost_empty: FIFO almost
  empty indicator (programmable threshold), high effective.",
  "specification": "- Support IEEE 754 Half-Precision Floating-Point Format (FP16);\n Support
  asynchronous clock domains (wr_clk and rd_clk are independent, with arbitrary frequencies and
  phases);\n Use Gray code-encoded read and write pointers, synchronized across clock domains
  via two-stage synchronizers to avoid metastability;\n FIFO depth is 16 (can store 16 16-bit data
  words). Programmable almost-full threshold (default: 14) and almost-empty threshold (default:
  2);\n Asynchronous reset behavior: When rst_n is low, all read/write pointers are cleared,
  full/empty/almost_full/almost_empty flags are reset, and the FIFO is initialized to empty;\n Data
  write: When wr_en is high and full is low, data_in is written into the FIFO on the rising edge of
  wr_clk. Never perform read operations when empty (causes pointer misalignment);\n Data read:
  When rd_en is high and empty is low, data_out is read from the FIFO on the rising edge of rd_clk,
  following first-in-first-out (FIFO) order;\n Full flag generation: Asserted in the write clock domain
  when the FIFO is completely full (write pointer catches up to the synchronized read pointer in
  Gray code);\n Empty flag generation: Asserted in the read clock domain when the FIFO is
  completely empty (read pointer catches up to the synchronized write pointer in Gray code);\n
  Almost-full flag: Asserted when the number of occupied entries reaches the almost-full threshold
  (programmable, default 14);\n Almost-empty flag: Asserted when the number of occupied
  entries drops to the almost-empty threshold (programmable, default 2);\n Traditional Verilog
  has limited support for loop-based assignments to reg arrays. The syntax of iterating through an
  array and assigning values via a for loop within an always block is not allowed in older standards.
  Therefore, if you need to initialize and clear the memory of this FIFO, you either place the
  initialization in an initial block (for simulation purposes) or use other methods to clear the
  memory;\n -The synchronizer delay from the write pointer to the read pointer in the
  asynchronous FIFO causes the read domain to fail to timely detect that the FIFO is not empty,
  thus losing the first written data. Therefore, it is necessary to ensure that the start time of the
  read operation is later than the synchronizer's stable time, or to optimize the synchronization
  logic to reduce the impact of delay on the critical path;\n Read and write pointers use n+1 bits
  (e.g., 5 bits for a depth of 16, where the highest 1 bit serves as the carry bit and the lower 4 bits
  as the address). When calculating FIFO counts (fifo_count_wr_domain/fifo_count_rd_domain), it
  is mandatory to check whether the highest 1 bit of the pointers is consistent: 1. If the highest 1 bit
  is the same, the count equals the difference of the lower n bits; 2. If the highest 1 bit is different
  (pointer wrap-around), the count equals the difference of the lower n bits plus the depth.
  Directly calculating the count using wr_ptr_bin - rd_ptr_bin_sync or similar methods is prohibited

```

```

(failure to handle wrap-around will result in errors in full/empty flags)".
"rtl_code": "module asyn_fifo #(
  parameter ALMOST_FULL_THRESHOLD = 14,
  parameter ALMOST_EMPTY_THRESHOLD = 2) (
  input wire wr_clk,
  input wire rd_clk,
  input wire rst_n,
  input wire wr_en,
  input wire rd_en,
  input [15:0] data_in,
  output reg [15:0] data_out,
  output reg full,
  output reg empty,
  output reg almost_full,
  output reg almost_empty) (
  // FIFO depth
  and pointer parameters
  localparam DEPTH = 16,
  localparam ADDR_WIDTH = 4,
  localparam PTR_WIDTH = ADDR_WIDTH + 1,
  // Memory array
  reg [15:0]
  mem [0:DEPTH-1],
  // Pointers in binary and gray code
  reg [PTR_WIDTH-1:0]
  wr_ptr_bin, rd_ptr_bin,
  reg [PTR_WIDTH-1:0] wr_ptr_gray, rd_ptr_gray,
  // Synchronized pointers
  reg [PTR_WIDTH-1:0] wr_ptr_gray_sync, rd_ptr_gray_sync,
  reg [PTR_WIDTH-1:0] wr_ptr_gray_sync2, rd_ptr_gray_sync2,
  wire [PTR_WIDTH-1:0]
  wr_ptr_bin_sync, rd_ptr_bin_sync,
  // Binary to Gray conversion
  function
  [PTR_WIDTH-1:0] bin2gray,
  input [PTR_WIDTH-1:0] bin,
  output [PTR_WIDTH-1:0] gray,
  (bin >> 1);
  endfunction
  // Gray to Binary conversion
  function
  [PTR_WIDTH-1:0] gray2bin,
  input [PTR_WIDTH-1:0] gray,
  output [PTR_WIDTH-1:0] bin,
  integer i;
  begin
    bin[PTR_WIDTH-1] = gray[PTR_WIDTH-1];
    for (i = PTR_WIDTH-2; i >= 0; i = i - 1) begin
      bin[i] = bin[i+1] ^ gray[i];
    end
    gray2bin = bin;
  end
endfunction
// Convert synchronized Gray codes back to binary
assign rd_ptr_bin_sync =
  gray2bin(wr_ptr_gray_sync2);
assign rd_ptr_bin_sync =
  gray2bin(rd_ptr_gray_sync2);
// Two-stage synchronizers
always @(posedge
  rd_clk or negedge rst_n) begin
  if (!rst_n) begin
    wr_ptr_gray_sync <=
      (PTR_WIDTH[1'b0]);
    wr_ptr_gray_sync2 <=
      (PTR_WIDTH[1'b0]);
  end
  else begin
    wr_ptr_gray_sync <=
      wr_ptr_gray;
    wr_ptr_gray_sync2 <=
      wr_ptr_gray;
  end
end
always
  @(posedge wr_clk or negedge rst_n) begin
    if (!rst_n) begin
      rd_ptr_gray_sync <=
        (PTR_WIDTH[1'b0]);
      rd_ptr_gray_sync2 <=
        (PTR_WIDTH[1'b0]);
    end
    else begin
      rd_ptr_gray_sync <=
        rd_ptr_gray;
      rd_ptr_gray_sync2 <=
        rd_ptr_gray;
    end
  end
  // Write pointer and memory control
  always @(posedge wr_clk or
  negedge rst_n) begin
    if (!rst_n) begin
      wr_ptr_bin <=
        (PTR_WIDTH[1'b0]);
    end
    else if
      (wr_en && !full) begin
        mem[wr_ptr_bin[ADDR_WIDTH-1:0]] <= data_in;
        wr_ptr_bin <= wr_ptr_bin + 1'b1;
        wr_ptr_gray <= bin2gray(wr_ptr_bin + 1'b1);
      end
  end
  // Read pointer and output control
  always @(posedge rd_clk
  or negedge rst_n) begin
    if (!rst_n) begin
      rd_ptr_bin <=
        (PTR_WIDTH[1'b0]);
    end
    else if (rd_en && !empty) begin
      data_out <=
        mem[rd_ptr_bin[ADDR_WIDTH-1:0]];
      rd_ptr_bin <= rd_ptr_bin + 1'b1;
      rd_ptr_gray <= bin2gray(rd_ptr_bin + 1'b1);
    end
  end
  // Flag
  generation in write domain
  always @(posedge wr_clk or negedge rst_n) begin
    if (!rst_n) begin
      full <= 1'b0;
      almost_full <= 1'b0;
    end
    else begin
      // Full when next write would make write pointer equal to

```

```

full=<rk>), empty, full)\n                error_count++;\n                end else begin\n$display("\tPASS: Reset Test!");\n                //Test 2: Basic Read-Write Test\nTestIn $display("\tTest 2: Basic Read-Write Test");\n                write_count = 0;\n                rd_count = 0;\n                // Write 8 data items\n                for (inti = 0; i < 8; i++)\nbegin\n                @($posedge wr_clk)\n                wr_en = 1;\n                data_in =\n                test_data[i];\n                @($posedge wr_clk)\n                #1;\n                wr_en =\n                0;\n                write_count++; end\n                // #200; // Longer wait\nfor synchronization\n                // Read 8 data items\n                for (inti = 0; i < 8; i++)\nbegin\n                @($negedge rd_clk)\n                rd_en = 1;\n                @($posedge rd_clk)\n                #2; // Wait for output to stabilize\n                received_data[i] = data_out;\n                if (data_out != test_data[i]) begin\n$display("\tERROR: Data mismatch at read %0d: expected=%h, got=%h", i, test_data[i],\n                data_out);\n                error_count++;\n                rd_count++;\n                rd_en = 0;\n                read_count++;\n                end\n                //Test 3: Full Condition TestIn $display("\tTest 3: Full\n                Condition Test");\n                // Reset and start fresh\n                rst_n = 0;\n                #200;\n                #200;\n                // Fill the FIFO\n                for (inti = 0; i < DEPTH; i++) begin\n                @($negedge wr_clk)\n                wr_en =\n                1;\n                data_in = test_data[i];\n                @($posedge wr_clk)\n                #1;\n                wr_en = 0;\n                // Wait a few more cycles\n                for full flag to update\n                repeat (4) @($posedge wr_clk);\n                // #200; // Wait for\n                synchronization\n                // Check full flag\n                if (full == 'b1) begin\n$display("\tERROR: Full flag not asserted after %0d writes", DEPTH);\n                error_count++;\n                end\n                // Try to write when full\n                data_in = 16'hFFFFFF;\n                @($negedge wr_clk)\n                wr_en = 1;\n                wr_en = 0;\n                // Should not\n                @($posedge wr_clk)\n                #1;\n                wr_en = 0;\n                // Should not\n                be able to write more data\n                if (full == 'b1) begin\n$display("\tERROR:\n                Full condition violated");\n                error_count++;\n                end else begin\n$display("\tPASS: Full Condition Test");\n                end\n                //Test 4: Empty\n                Condition TestIn $display("\tTest 4: Empty Condition Test");\n                // Reset and start fresh\n                rst_n = 0;\n                #200;\n                // #200;\n                // Empty the FIFO\n                for (inti = 0; i < DEPTH; i++) begin\n                @($negedge rd_clk)\n                rd_en = 1;\n                @($posedge rd_clk)\n                #2;\n                @($negedge rd_clk)\n                rd_en = 0;\n                // Check empty flag\n                if\n                (empty == 'b1) begin\n$display("\tERROR: Empty flag not asserted");\n                error_count++;\n                end\n                @($posedge wr_clk)\n                #1;\n                wr_en = 1;\n                data_in =\n                test_data[i];\n                @($posedge wr_clk)\n                #1;\n                wr_en = 0;\n                //Test 5: Almost-Full/Almost-Empty Threshold TestIn\n                TestIn $display("\tTest 5: Almost-Full/Almost-Empty Threshold Test");\n                // Reset and start fresh\n                rst_n = 0;\n                #200;\n                // #200;\n                // Write until almost full\n                for (inti = 0; i < 10; i++)\nbegin\n                @($negedge wr_clk)\n                wr_en = 1;\n                data_in =\n                test_data[i];\n                @($posedge wr_clk)\n                #1;\n                wr_en = 0;\n                //Test 6: Almost-Full/Almost-Empty Threshold TestIn\n                TestIn $display("\tTest 6: Almost-Full/Almost-Empty Threshold Test");\n                // Reset and start fresh\n                rst_n = 0;\n                #200;\n                // #200;\n                // Write until almost full\n                for (inti = 0; i < 10; i++)\nbegin\n                @($negedge wr_clk)\n                wr_en = 1;\n                data_in =\n                test_data[i];\n                @($posedge wr_clk)\n                #1;\n                wr_en = 0;\n                //Test 7: Almost-Full/Almost-Empty Threshold TestIn\n                TestIn $display("\tTest 7: Almost-Full/Almost-Empty Threshold Test");\n                // Reset and start fresh\n                rst_n = 0;\n                #200;\n                // #200;\n                // Write until almost full\n                for (inti = 0; i < 10; i++)\nbegin\n                @($negedge wr_clk)\n                wr_en = 1;\n                data_in =\n                test_data[i];\n                @($posedge wr_clk)\n                #1;\n                wr_en = 0;\n                //Test 8: Almost-Full/Almost-Empty Threshold TestIn\n                TestIn $display("\tTest 8: Almost-Full/Almost-Empty Threshold Test");\n                // Reset and start fresh\n                rst_n = 0;\n                #200;\n                // #200;\n                // Write until almost full\n                for (inti = 0; i < 10; i++)\nbegin\n                @($negedge wr_clk)\n                wr_en = 1;\n                data_in =\n                test_data[i];\n                @($posedge wr_clk)\n                #1;\n                wr_en = 0;\n                //Test 9: Almost-Full/Almost-Empty Threshold TestIn\n                TestIn $display("\tTest 9: Almost-Full/Almost-Empty Threshold Test");\n                // Reset and start fresh\n                rst_n = 0;\n                #200;\n                // #200;\n                // Write until almost full\n                for (inti = 0; i < 10; i++)\nbegin\n                @($negedge wr_clk)\n                wr_en = 1;\n                data_in =\n                test_data[i];\n                @($posedge wr_clk)\n                #1;\n                wr_en = 0;\n                //Test 10: Almost-Full/Almost-Empty Threshold TestIn\n                TestIn $display("\tTest 10: Almost-Full/Almost-Empty Threshold Test");\n                // Reset and start fresh\n                rst_n = 0;\n                #200;\n                // #200;\n                // Write until almost full\n                for (inti = 0; i < 10; i++)\nbegin\n                @($negedge wr_clk)\n                wr_en = 1;\n                data_in =\n                test_data[i];\n                @($posedge wr_clk)\n                #1;\n                wr_en = 0;\n                //Test 11: Almost-Full/Almost-Empty Threshold TestIn\n                TestIn $display("\tTest 11: Almost-Full/Almost-Empty Threshold Test");\n                // Reset and start fresh\n                rst_n = 0;\n                #200;\n                // #200;\n                // Write until almost full\n                for (inti = 0; i < 10; i++)\nbegin\n                @($negedge wr_clk)\n                wr_en = 1;\n                data_in =\n                test_data[i];\n                @($posedge wr_clk)\n                #1;\n                wr_en = 0;\n                //Test 12: Almost-Full/Almost-Empty Threshold TestIn\n                TestIn $display("\tTest 12: Almost-Full/Almost-Empty Threshold Test");\n                // Reset and start fresh\n                rst_n = 0;\n                #200;\n                // #200;\n                // Write until almost full\n                for (inti = 0; i < 10; i++)\nbegin\n                @($negedge wr_clk)\n                wr_en = 1;\n                data_in =\n                test_data[i];\n                @($posedge wr_clk)\n                #1;\n                wr_en = 0;\n                //Test 13: Almost-Full/Almost-Empty Threshold TestIn\n                TestIn $display("\tTest 13: Almost-Full/Almost-Empty Threshold Test");\n                // Reset and start fresh\n                rst_n = 0;\n                #200;\n                // #200;\n                // Write until almost full\n                for (inti = 0; i < 10; i++)\nbegin\n                @($negedge wr_clk)\n                wr_en = 1;\n                data_in =\n                test_data[i];\n                @($posedge wr_clk)\n                #1;\n                wr_en = 0;\n                //Test 14: Almost-Full/Almost-Empty Threshold TestIn\n                TestIn $display("\tTest 14: Almost-Full/Almost-Empty Threshold Test");\n                // Reset and start fresh\n                rst_n = 0;\n                #200;\n                // #200;\n                // Write until almost full\n                for (inti = 0; i < 10; i++)\nbegin\n                @($negedge wr_clk)\n                wr_en = 1;\n                data_in =\n                test_data[i];\n                @($posedge wr_clk)\n                #1;\n                wr_en = 0;\n                //Test 15: Almost-Full/Almost-Empty Threshold TestIn\n                TestIn $display("\tTest 15: Almost-Full/Almost-Empty Threshold Test");\n                // Reset and start fresh\n                rst_n = 0;\n                #200;\n                // #200;\n                // Write until almost full\n                for (inti = 0; i < 10; i++)\nbegin\n                @($negedge wr_clk)\n                wr_en = 1;\n                data_in =\n                test_data[i];\n                @($posedge wr_clk)\n                #1;\n                wr_en = 0;\n                //Test 16: Almost-Full/Almost-Empty Threshold TestIn\n                TestIn $display("\tTest 16: Almost-Full/Almost-Empty Threshold Test");\n                // Reset and start fresh\n                rst_n = 0;\n                #200;\n                // #200;\n                // Write until almost full\n                for (inti = 0; i < 10; i++)\nbegin\n                @($negedge wr_clk)\n                wr_en = 1;\n                data_in =\n                test_data[i];\n                @($posedge wr_clk)\n                #1;\n                wr_en = 0;\n                //Test 17: Almost-Full/Almost-Empty Threshold TestIn\n                TestIn $display("\tTest 17: Almost-Full/Almost-Empty Threshold Test");\n                // Reset and start fresh\n                rst_n = 0;\n                #200;\n                // #200;\n                // Write until almost full\n                for (inti = 0; i < 10; i++)\nbegin\n                @($negedge wr_clk)\n                wr_en = 1;\n                data_in =\n                test_data[i];\n                @($posedge wr_clk)\n                #1;\n                wr_en = 0;\n                //Test 18: Almost-Full/Almost-Empty Threshold TestIn\n                TestIn $display("\tTest 18: Almost-Full/Almost-Empty Threshold Test");\n                // Reset and start fresh\n                rst_n = 0;\n                #200;\n                // #200;\n                // Write until almost full\n                for (inti = 0; i < 10; i++)\nbegin\n                @($negedge wr_clk)\n                wr_en = 1;\n                data_in =\n                test_data[i];\n                @($posedge wr_clk)\n                #1;\n                wr_en = 0;\n                //Test 19: Almost-Full/Almost-Empty Threshold TestIn\n                TestIn $display("\tTest 19: Almost-Full/Almost-Empty Threshold Test");\n                // Reset and start fresh\n                rst_n = 0;\n                #200;\n                // #200;\n                // Write until almost full\n                for (inti = 0; i < 10; i++)\nbegin\n                @($negedge wr_clk)\n                wr_en = 1;\n                data_in =\n                test_data[i];\n                @($posedge wr_clk)\n                #1;\n                wr_en = 0;\n                //Test 20: Almost-Full/Almost-Empty Threshold TestIn\n                TestIn $display("\tTest 20: Almost-Full/Almost-Empty Threshold Test");\n                // Reset and start fresh\n                rst_n = 0;\n                #200;\n                // #200;\n                // Write until almost full\n                for (inti = 0; i < 10; i++)\nbegin\n                @($negedge wr_clk)\n                wr_en = 1;\n                data_in =\n                test_data[i];\n                @($posedge wr_clk)\n                #1;\n                wr_en = 0;\n                //Test 21: Almost-Full/Almost-Empty Threshold TestIn\n                TestIn $display("\tTest 21: Almost-Full/Almost-Empty Threshold Test");\n                // Reset and start fresh\n                rst_n = 0;\n                #200;\n                // #200;\n                // Write until almost full\n                for (inti = 0; i < 10; i++)\nbegin\n                @($negedge wr_clk)\n                wr_en = 1;\n                data_in =\n                test_data[i];\n                @($posedge wr_clk)\n                #1;\n                wr_en = 0;\n                //Test 22: Almost-Full/Almost-Empty Threshold TestIn\n                TestIn $display("\tTest 22: Almost-Full/Almost-Empty Threshold Test");\n                // Reset and start fresh\n                rst_n = 0;\n                #200;\n                // #200;\n                // Write until almost full\n                for (inti = 0; i < 10; i++)\nbegin\n                @($negedge wr_clk)\n                wr_en = 1;\n                data_in =\n                test_data[i];\n                @($posedge wr_clk)\n                #1;\n                wr_en = 0;\n                //Test 23: Almost-Full/Almost-Empty Threshold TestIn\n                TestIn $display("\tTest 23: Almost-Full/Almost-Empty Threshold Test");\n                // Reset and start fresh\n                rst_n = 0;\n                #200;\n                // #200;\n                // Write until almost full\n                for (inti = 0; i < 10; i++)\nbegin\n                @($negedge wr_clk)\n                wr_en = 1;\n                data_in =\n                test_data[i];\n                @($posedge wr_clk)\n                #1;\n                wr_en = 0;\n                //Test 24: Almost-Full/Almost-Empty Threshold TestIn\n                TestIn $display("\tTest 24: Almost-Full/Almost-Empty Threshold Test");\n                // Reset and start fresh\n                rst_n = 0;\n                #200;\n                // #200;\n                // Write until almost full\n                for (inti = 0; i < 10; i++)\nbegin\n                @($negedge wr_clk)\n                wr_en = 1;\n                data_in =\n                test_data[i];\n                @($posedge wr_clk)\n                #1;\n                wr_en = 0;\n                //Test 25: Almost-Full/Almost-Empty Threshold TestIn\n                TestIn $display("\tTest 25: Almost-Full/Almost-Empty Threshold Test");\n                // Reset and start fresh\n                rst_n = 0;\n                #200;\n                // #200;\n                // Write until almost full\n                for (inti = 0; i < 10; i++)\nbegin\n                @($negedge wr_clk)\n                wr_en = 1;\n                data_in =\n
```

```
Wait for synchronization\n                                \\n                                if (almost_empty != 1'b1) begin\\n$display(\\\"ERROR: Edge case - almost empty flag not asserted\\\");\\nerror_count++;\\nend\\n                                // Read all remaining entries (empty())\\nfor (int i = 0; i < 3; i++) begin\\n                                @(!negedge rd_clk);\\n                                rd_en = 1;\\n@(!posedge rd_clk);\\n                                @(!negedge rd_clk);\\nrd_en = 0;\\nend\\n                                // #200; // Wait for synchronization\\n\\n                                $display(\\\"ERROR: Edge case - empty flag not asserted\\\");\\n                                error_count++;\\n                                end else begin\\n$display(\\\"PASS: Edge Case Test\\\");\\n                                end\\n                                // Final result\\nif (error_count == 0) begin\\n                                $display(\\\"TestSucceeded: All tests passed\\\");\\nend else begin\\n                                $display(\\\"TestFailed: %0d errors found\\\", error_count);\\nend\\n                                $finish;\\n                                end\\n                                // Monitor\\n$monitor(\\\"Timer=%0t, wr_en=%b, rd_en=%b, data_in=%h, data_out=%h, full=%b, empty=%b\\\", almost_full, almost_empty, \\n                                $time, wr_en, rd_en, data_in, data_out, full, almost_full, almost_empty);\\n                                end\\n\\nendmodule\\\"\\n\\n    \"test_spec\": \"- Basic Read-Write Test: Write a sequence of data (e.g., 0x0000 to 0x00FF) and read them back, verifying FIFO order and data integrity.\\n- Asynchronous Clock Test: Configure wr_clk at 100MHz and rd_clk at 50MHz, perform continuous write and read operations, verifying data correctness across clock domains.\\n- Full Condition Test: Continuously write data until full is asserted, check that no more data can be written, and verify full flag accuracy.\\n- Empty Condition Test: Continuously read data until empty is asserted, check that no more data can be read, and verify empty flag accuracy.\\n- Almost-Full/Almost-Empty Threshold Test: Set custom thresholds (e.g., almost_full=10, almost_empty=3), verify that the flags are asserted correctly when the entry count reaches the thresholds.\\n- Reset Test: Assert rd_n during operation, check that all pointers, flags, and data outputs are initialized correctly after reset.\\n- Concurrent Read-Write Test: Perform write and read operations simultaneously (with proper clock domain handling), verify FIFO stability and data consistency.\\n- Edge Case Test: Verify 15 entries (one before full), verify 16 entries (full), read 1 entry (almost empty at threshold), read all entries (empty), and verify corresponding flag behaviors.\\n- Once a test case passes, please print it out.\\n- Ensure wr_en is asserted after the wr_clk negedge (falling edge) and remains high until the next wr_clk posedge (rising edge). This guarantees wr_en has sufficient setup time (at least half the wr_clk period) before the rising edge, so the write logic reliably detects wr_en=1 on the posedge.\"\\n\\n    \"RelatedDo\": [\\n        1,\\n        2,\\n        3,\\n        4,\\n        5,\\n        8,\\n        9,\\n        10,\\n        12,\\n        15,\\n    ],\\n    \"SimulationResult\": \"\\nStarting Async FIFO Test...\\nTest 1: Reset Test\\nTime:0t: wr_en=0
```

[illegible]

```

almost_empty=1\vr_en=0, rd_en=1_data_in=ffff_data_out=0000, full=empty=1
almost_full=1\vr_en=1\vr_time=2540_wr_en=0, rd_en=0_data_in=ffff_data_out=0000
full=empty=1, almost_full=0, almost_empty=1\vr_time=2560_wr_en=0, rd_en=1_data_in=ffff
data_out=0000, full=0, empty=1, almost_full=0, almost_empty=1\vr_time=2580_wr_en=0
rd_en=0, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
almost_empty=1\vr_time=2600_wr_en=0, rd_en=1_data_in=ffff_data_out=0000, full=empty=1
almost_full=0, almost_empty=1\vr_time=2620_wr_en=0, rd_en=0_data_in=ffff_data_out=0000
full=0, empty=1, almost_full=0, almost_empty=1\vrPASS_Empty Condition Test\vr_nTest
Almost-Full/Almost-Empty Threshold Test\vr_time=3220_wr_en=1, rd_en=0_data_in=0000
data_out=0000, full=0, empty=1, almost_full=0, almost_empty=1\vr_time=3236_wr_en=0
rd_en=0, data_in=0000, data_out=0000, full=0, empty=1, almost_full=0,
almost_empty=1\vr_time=3230_wr_en=1, rd_en=0_data_in=0001, data_out=0000, full=0,
empty=1, almost_full=0, almost_empty=1\vr_time=3236_wr_en=0, rd_en=0_data_in=0001,
data_out=0000, full=0, empty=1, almost_full=0, almost_empty=1\vr_time=3240_wr_en=1
rd_en=0, data_in=0002, data_out=0000, full=0, empty=1, almost_full=0,
almost_empty=1\vr_time=3246_wr_en=0, rd_en=0, data_in=0002, data_out=0000, full=0,
empty=1, almost_full=0, almost_empty=1\vr_time=3250_wr_en=1, rd_en=0_data_in=0003
data_out=0000, full=0, empty=1, almost_full=0, almost_empty=1\vr_time=3256_wr_en=0
rd_en=0, data_in=0003, data_out=0000, full=0, empty=1, almost_full=0,
almost_empty=1\vr_time=3260_wr_en=1, rd_en=0_data_in=0004, data_out=0000, full=0,
empty=1, almost_full=0, almost_empty=1\vr_time=3266_wr_en=0, rd_en=0_data_in=0004
data_out=0000, full=0, empty=1, almost_full=0, almost_empty=1\vr_time=3270_wr_en=1
rd_en=0, data_in=0005, data_out=0000, full=0, empty=0, almost_full=0,
almost_empty=1\vr_time=3276_wr_en=0, rd_en=0_data_in=0005, data_out=0000, full=0,
empty=0, almost_full=0, almost_empty=1\vr_time=3280_wr_en=1, rd_en=0_data_in=0006
data_out=0000, full=0, empty=0, almost_full=0, almost_empty=1\vr_time=3286_wr_en=0
rd_en=0, data_in=0006, data_out=0000, full=0, empty=0, almost_full=0,
almost_empty=1\vr_time=3290_wr_en=1, rd_en=0_data_in=0007, data_out=0000, full=0,
empty=0, almost_full=0, almost_empty=1\vr_time=3296_wr_en=0, rd_en=0_data_in=0007
data_out=0000, full=0, empty=0, almost_full=0, almost_empty=1\vr_time=3300_wr_en=1
rd_en=0, data_in=0008, data_out=0000, full=0, empty=0, almost_full=0,
almost_empty=1\vr_time=3306_wr_en=0, rd_en=0_data_in=0008, data_out=0000, full=0,
empty=0, almost_full=0, almost_empty=1\vr_time=3310_wr_en=1, rd_en=0_data_in=0009
data_out=0000, full=0, empty=0, almost_full=0, almost_empty=0\vr_time=3316_wr_en=0
rd_en=0, data_in=0009, data_out=0000, full=0, empty=0, empty=0, almost_full=0,
almost_empty=0\vr_time=3325_wr_en=0, rd_en=0_data_in=0009, data_out=0000, full=0,
empty=0, almost_full=0, almost_empty=0\vr_time=3520_wr_en=0, rd_en=1_data_in=0009
data_out=0000, full=0, empty=0, almost_full=1, almost_empty=0\vr_time=3540_wr_en=0
rd_en=0, data_in=0009, data_out=0000, full=0, empty=0, almost_full=1,
almost_empty=0\vr_time=3555_wr_en=0, rd_en=0_data_in=0009, data_out=0000, full=0,
empty=0, almost_full=0, almost_empty=0\vr_time=3560_wr_en=0, rd_en=1_data_in=0009
data_out=0000, full=0, empty=0, almost_full=0, almost_empty=0\vr_time=3570_wr_en=0
rd_en=1, data_in=0009, data_out=0001, full=0, empty=0, almost_full=0,
almost_empty=0\vr_time=3580_wr_en=0, rd_en=0_data_in=0009, data_out=0001, full=0,

```

[illegible][illegible][illegible][illegible]


```

rd_en = 1;
1'b1 begin
error_count++;
end
task threshold_test;
begin
// Test almost_full threshold
for (int i = 0; i < ALMOST_FULL_THRESH; i++) begin
@posedge wr_clk;
wr_en = 1;
data_in = test_data[i];
#1;
end
if (almost_full == 1'b1) begin
$display("ERROR: Almost_full not asserted at threshold");
error_count++;
end
// Test almost_empty threshold
ALMOST_EMPTY_THRESH; i++) begin
@posedge rd_clk;
rd_en = 1;
end
if (almost_empty == 1'b1) begin
$display("ERROR: Almost_empty not asserted at threshold");
error_count++;
end
endtask
concurrent_test;
begin
// Reset and start fresh
rst_n = 0;
#20;
rst_n = 1;
#20;
fork
// Writer thread
for (int i = 0; i < 32; i++) begin
@posedge wr_clk;
wr_en = 1;
data_in = test_data[i] % 16;
end
// Reader thread
begin
#50; // Start reading after some delay
for (int i = 0; i < 32; i++) begin
@posedge rd_clk;
rd_en = 1;
// Verify
if (empty && data_out != test_data[i] % 16)
$display("ERROR: Concurrent test data mismatch at read %0d", i);
error_count++;
end
end
join
endtask
task edge_case_test;
begin
// Reset and start fresh
rst_n = 0;
#20;
rst_n = 1;
// Write 15 entries (one before full)
for (int i = 0; i < 15; i++) begin
@posedge wr_clk;
wr_en = 1;
data_in = test_data[i];
end
// Check flags
if (full == 1'b0 || almost_full == 1'b1) begin
$display("ERROR: Flags incorrect after 15 writes");
error_count++;
end
// Write one more to make it full
@posedge wr_clk;
wr_en = 1;
data_in = test_data[15];
end
if (full == 1'b1) begin
$display("ERROR: Full flag not asserted after 16 writes");
error_count++;
end
// Read one entry
@posedge rd_clk;
rd_en = 1;
end
endtask
endmodule

```

```

@posedge rd_clk;
#1;
rd_en = 0;
if (almost_empty == 1'b1) begin
$display("ERROR: Almost_empty not asserted after reading from full");
error_count++;
end
// Read all remaining entries
for (int i = 0; i < 15; i++) begin
@posedge rd_clk;
rd_en = 1;
end
if (empty == 1'b1) begin
$display("ERROR: Empty flag not asserted after reading all");
error_count++;
end
endtask
endmodule

```

2 最终设计出来的 RTL 代码和 TestBench

整个 Agent 的工作流程如图:

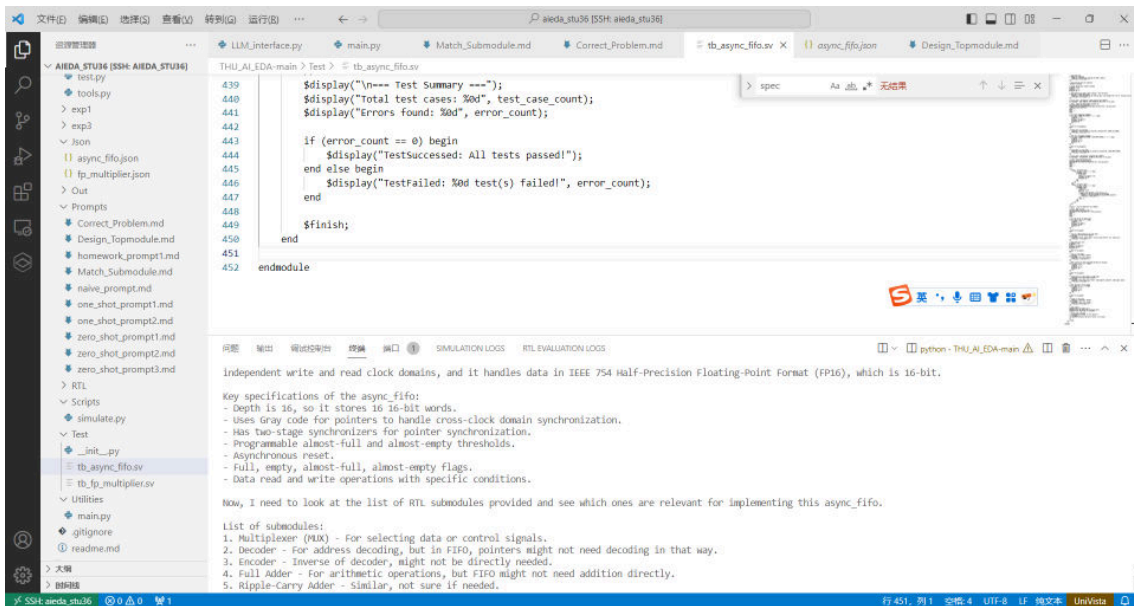


图 1: 子模块匹配

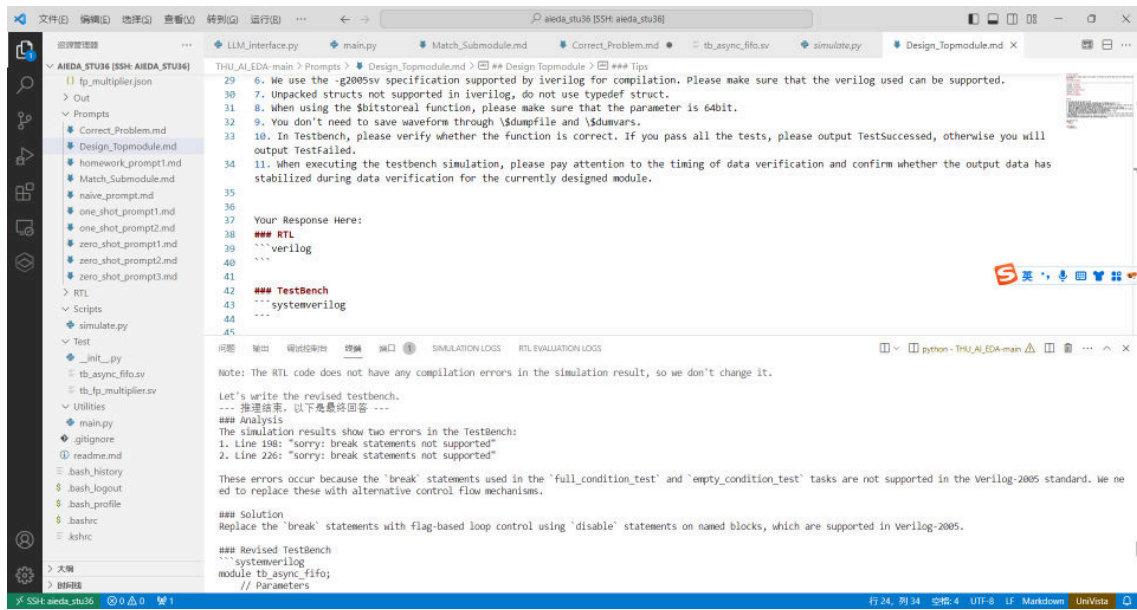


图 2: 代码迭代纠正

```

1 module async_fifo #(
2     parameter ALMOST_FULL_THRESHOLD = 14,
3     parameter ALMOST_EMPTY_THRESHOLD = 2
4 ) (
5     input wire wr_clk,
6     input wire rd_clk,
7     input wire rst_n,
8     input wire wr_en,
9     input wire rd_en,
10    input wire [15:0] data_in,
11    output reg [15:0] data_out,
12    output reg full,
13    output reg empty,
14    output reg almost_full,
15    output reg almost_empty
16 );
17
18    // FIFO depth and pointer parameters
19    localparam DEPTH = 16;
20    localparam ADDR_WIDTH = 4;
21    localparam PTR_WIDTH = ADDR_WIDTH + 1;
22
23    // Memory array
24    reg [15:0] mem [0:DEPTH-1];
25
26    // Pointers in binary and gray code
27    reg [PTR_WIDTH-1:0] wr_ptr_bin, rd_ptr_bin;
28    reg [PTR_WIDTH-1:0] wr_ptr_gray, rd_ptr_gray;
29
30    // Synchronized pointers
31    reg [PTR_WIDTH-1:0] wr_ptr_gray_sync, rd_ptr_gray_sync;
32    reg [PTR_WIDTH-1:0] wr_ptr_gray_sync2, rd_ptr_gray_sync2;
33    wire [PTR_WIDTH-1:0] wr_ptr_bin_sync, rd_ptr_bin_sync;
34
35    // Binary to Gray conversion
36    function [PTR_WIDTH-1:0] bin2gray;
37        input [PTR_WIDTH-1:0] bin;
38        bin2gray = bin ^ (bin >> 1);

```

```

39     endfunction
40
41     // Gray to Binary conversion
42     function [PTR_WIDTH-1:0] gray2bin;
43         input [PTR_WIDTH-1:0] gray;
44         reg [PTR_WIDTH-1:0] bin;
45         integer i;
46         begin
47             bin[PTR_WIDTH-1] = gray[PTR_WIDTH-1];
48             for (i = PTR_WIDTH-2; i >= 0; i = i - 1) begin
49                 bin[i] = bin[i+1] ^ gray[i];
50             end
51             gray2bin = bin;
52         end
53     endfunction
54
55     // Convert synchronized Gray codes back to binary
56     assign wr_ptr_bin_sync = gray2bin(wr_ptr_gray_sync2);
57     assign rd_ptr_bin_sync = gray2bin(rd_ptr_gray_sync2);
58
59     // Two-stage synchronizers
60     always @(posedge rd_clk or negedge rst_n) begin
61         if (!rst_n) begin
62             wr_ptr_gray_sync <= {PTR_WIDTH{1'b0}};
63             wr_ptr_gray_sync2 <= {PTR_WIDTH{1'b0}};
64         end else begin
65             wr_ptr_gray_sync <= wr_ptr_gray;
66             wr_ptr_gray_sync2 <= wr_ptr_gray_sync;
67         end
68     end
69
70     always @(posedge wr_clk or negedge rst_n) begin
71         if (!rst_n) begin
72             rd_ptr_gray_sync <= {PTR_WIDTH{1'b0}};
73             rd_ptr_gray_sync2 <= {PTR_WIDTH{1'b0}};
74         end else begin
75             rd_ptr_gray_sync <= rd_ptr_gray;
76             rd_ptr_gray_sync2 <= rd_ptr_gray_sync;
77         end
78     end
79
80     // Write pointer and memory control
81     always @(posedge wr_clk or negedge rst_n) begin
82         if (!rst_n) begin
83             wr_ptr_bin <= {PTR_WIDTH{1'b0}};
84             wr_ptr_gray <= {PTR_WIDTH{1'b0}};
85         end else if (wr_en && !full) begin
86             mem[wr_ptr_bin[ADDR_WIDTH-1:0]] <= data_in;
87             wr_ptr_bin <= wr_ptr_bin + 1'b1;
88             wr_ptr_gray <= bin2gray(wr_ptr_bin + 1'b1);
89         end
90     end
91
92     // Read pointer and output control
93     always @(posedge rd_clk or negedge rst_n) begin
94         if (!rst_n) begin
95             rd_ptr_bin <= {PTR_WIDTH{1'b0}};
96             rd_ptr_gray <= {PTR_WIDTH{1'b0}};
97             data_out <= 16'b0;
98         end else if (rd_en && !empty) begin
99             data_out <= mem[rd_ptr_bin[ADDR_WIDTH-1:0]];
100            rd_ptr_bin <= rd_ptr_bin + 1'b1;

```

```

101         rd_ptr_gray <= bin2gray(rd_ptr_bin + 1'b1);
102     end
103 end
104
105 // Flag generation in write domain
106 always @(posedge wr_clk or negedge rst_n) begin
107     if (!rst_n) begin
108         full <= 1'b0;
109         almost_full <= 1'b0;
110     end else begin
111         // Full when next write would make write pointer equal to synchronized read
112         pointer
113         if (wr_en && !full) begin
114             full <= ((wr_ptr_bin + 1'b1) == {~rd_ptr_bin_sync[PTR_WIDTH-1],
115 rd_ptr_bin_sync[ADDR_WIDTH-1:0]});
116         end else begin
117             full <= (wr_ptr_bin == {~rd_ptr_bin_sync[PTR_WIDTH-1], rd_ptr_bin_sync[
118 ADDR_WIDTH-1:0]});
119         end
120
121         // Calculate FIFO count for almost_full
122         if (wr_ptr_bin[PTR_WIDTH-1] == rd_ptr_bin_sync[PTR_WIDTH-1]) begin
123             almost_full <= ((wr_ptr_bin[ADDR_WIDTH-1:0] - rd_ptr_bin_sync[ADDR_WIDTH-1:0])
124 >= ALMOST_FULL_THRESHOLD);
125         end else begin
126             almost_full <= ((DEPTH - rd_ptr_bin_sync[ADDR_WIDTH-1:0] + wr_ptr_bin[
127 ADDR_WIDTH-1:0]) >= ALMOST_FULL_THRESHOLD);
128         end
129     end
130 end
131
132 // Flag generation in read domain
133 always @(posedge rd_clk or negedge rst_n) begin
134     if (!rst_n) begin
135         empty <= 1'b1;
136         almost_empty <= 1'b1;
137     end else begin
138         // Empty when read pointer equals synchronized write pointer
139         empty <= (rd_ptr_bin == wr_ptr_bin_sync);
140
141         // Calculate FIFO count for almost_empty
142         if (wr_ptr_bin_sync[PTR_WIDTH-1] == rd_ptr_bin[PTR_WIDTH-1]) begin
143             almost_empty <= ((wr_ptr_bin_sync[ADDR_WIDTH-1:0] - rd_ptr_bin[ADDR_WIDTH
144 -1:0]) <= ALMOST_EMPTY_THRESHOLD);
145         end else begin
146             almost_empty <= ((DEPTH - rd_ptr_bin[ADDR_WIDTH-1:0] + wr_ptr_bin_sync[
147 ADDR_WIDTH-1:0]) <= ALMOST_EMPTY_THRESHOLD);
148         end
149     end
150 end
151
152 // Memory initialization for simulation
153 initial begin
154     for (int i = 0; i < DEPTH; i = i + 1) begin
155         mem[i] = 16'b0;
156     end
157 end
158 endmodule

```

Listing 4: async_fifo.v

```

1 module tb_async_fifo;
2     reg wr_clk, rd_clk, rst_n;
3     reg wr_en, rd_en;
4     reg [15:0] data_in;
5     wire [15:0] data_out;
6     wire full, empty, almost_full, almost_empty;
7
8     // Test parameters
9     localparam DEPTH = 16;
10    localparam TEST_DATA_COUNT = 20;
11    reg [15:0] test_data [0:TEST_DATA_COUNT-1];
12    reg [15:0] received_data [0:TEST_DATA_COUNT-1];
13    integer write_count = 0;
14    integer read_count = 0;
15    integer error_count = 0;
16
17    // Instantiate DUT
18    async_fifo #(
19        .ALMOST_FULL_THRESHOLD(10),
20        .ALMOST_EMPTY_THRESHOLD(3)
21    ) dut (
22        .wr_clk(wr_clk),
23        .rd_clk(rd_clk),
24        .rst_n(rst_n),
25        .wr_en(wr_en),
26        .rd_en(rd_en),
27        .data_in(data_in),
28        .data_out(data_out),
29        .full(full),
30        .empty(empty),
31        .almost_full(almost_full),
32        .almost_empty(almost_empty)
33    );
34
35    // Clock generation
36    initial begin
37        wr_clk = 0;
38        forever #5 wr_clk = ~wr_clk; // 100MHz
39    end
40
41    initial begin
42        rd_clk = 0;
43        forever #10 rd_clk = ~rd_clk; // 50MHz
44    end
45
46    // Test data initialization
47    initial begin
48        for (int i = 0; i < TEST_DATA_COUNT; i++) begin
49            test_data[i] = i;
50        end
51    end
52
53    // Main test sequence
54    initial begin
55        $display("Starting Async FIFO Test...");
56        error_count = 0;
57
58        // Test 1: Reset Test
59        $display("Test 1: Reset Test");
60        rst_n = 0;
61        wr_en = 0;

```

```

62     rd_en = 0;
63     data_in = 0;
64     #100; // Longer reset period for synchronization
65     rst_n = 1;
66     #100;
67
68     // Check reset values
69     if (empty !== 1'b1 || full !== 1'b0) begin
70         $display("ERROR: Reset test failed - empty=%b, full=%b", empty, full);
71         error_count++;
72     end else begin
73         $display("PASS: Reset Test");
74     end
75
76     // Test 2: Basic Read-Write Test
77     $display("Test 2: Basic Read-Write Test");
78     write_count = 0;
79     read_count = 0;
80
81     // Write 8 data items
82     for (int i = 0; i < 8; i++) begin
83         @(negedge wr_clk);
84         wr_en = 1;
85         data_in = test_data[i];
86         @(posedge wr_clk);
87         #1;
88         wr_en = 0;
89         write_count++;
90     end
91
92     #200; // Longer wait for synchronization
93
94     // Read 8 data items
95     for (int i = 0; i < 8; i++) begin
96         @(negedge rd_clk);
97         rd_en = 1;
98         @(posedge rd_clk);
99         #2; // Wait for output to stabilize
100        received_data[i] = data_out;
101        if (data_out !== test_data[i]) begin
102            $display("ERROR: Data mismatch at read %0d: expected=%h, got=%h", i, test_data
[i], data_out);
103            error_count++;
104        end
105        @(negedge rd_clk);
106        rd_en = 0;
107        read_count++;
108    end
109
110    if (error_count == 0) begin
111        $display("PASS: Basic Read-Write Test");
112    end
113
114    // Test 3: Full Condition Test
115    $display("Test 3: Full Condition Test");
116
117    // Reset and start fresh
118    rst_n = 0;
119    #200;
120    rst_n = 1;
121    #200;
122

```

```

123 // Fill the FIFO
124 for (int i = 0; i < DEPTH; i++) begin
125     @(negedge wr_clk);
126     wr_en = 1;
127     data_in = test_data[i];
128     @(posedge wr_clk);
129     #1;
130     wr_en = 0;
131 end
132
133 // Wait a few more cycles for full flag to update
134 repeat (4) @(posedge wr_clk);
135 #200; // Wait for synchronization
136
137 // Check full flag
138 if (full !== 1'b1) begin
139     $display("ERROR: Full flag not asserted after %0d writes", DEPTH);
140     error_count++;
141 end
142
143 // Try to write when full
144 @(negedge wr_clk);
145 wr_en = 1;
146 data_in = 16'hFFFF;
147 @(posedge wr_clk);
148 #1;
149 wr_en = 0;
150
151 // Should not be able to write more data
152 if (full !== 1'b1) begin
153     $display("ERROR: Full condition violated");
154     error_count++;
155 end else begin
156     $display("PASS: Full Condition Test");
157 end
158
159 // Test 4: Empty Condition Test
160 $display("Test 4: Empty Condition Test");
161
162 // Reset and start fresh
163 rst_n = 0;
164 #200;
165 rst_n = 1;
166 #200;
167
168 // Empty the FIFO
169 for (int i = 0; i < DEPTH; i++) begin
170     @(negedge rd_clk);
171     rd_en = 1;
172     @(posedge rd_clk);
173     #2;
174     @(negedge rd_clk);
175     rd_en = 0;
176 end
177
178 #200; // Wait for synchronization
179
180 // Check empty flag
181 if (empty !== 1'b1) begin
182     $display("ERROR: Empty flag not asserted");
183     error_count++;
184 end else begin

```

```

185     $display("PASS: Empty Condition Test");
186 end
187
188 // Test 5: Almost-Full/Almost-Empty Threshold Test
189 $display("Test 5: Almost-Full/Almost-Empty Threshold Test");
190
191 // Reset and start fresh
192 rst_n = 0;
193 #200;
194 rst_n = 1;
195 #200;
196
197 // Write until almost full
198 for (int i = 0; i < 10; i++) begin
199     @(negedge wr_clk);
200     wr_en = 1;
201     data_in = test_data[i];
202     @(posedge wr_clk);
203     #1;
204     wr_en = 0;
205 end
206
207 #200; // Wait for synchronization
208
209 if (almost_full !== 1'b1) begin
210     $display("ERROR: Almost-full flag not asserted at threshold 10");
211     error_count++;
212 end
213
214 // Read until almost empty
215 for (int i = 0; i < 7; i++) begin
216     @(negedge rd_clk);
217     rd_en = 1;
218     @(posedge rd_clk);
219     #2;
220     @(negedge rd_clk);
221     rd_en = 0;
222 end
223
224 #200; // Wait for synchronization
225
226 if (almost_empty !== 1'b1) begin
227     $display("ERROR: Almost-empty flag not asserted at threshold 3");
228     error_count++;
229 end else begin
230     $display("PASS: Almost-Full/Almost-Empty Threshold Test");
231 end
232
233 // Test 6: Concurrent Read-Write Test
234 $display("Test 6: Concurrent Read-Write Test");
235
236 // Reset and start fresh
237 rst_n = 0;
238 #200;
239 rst_n = 1;
240 #200;
241
242 fork
243     // Writer thread
244     begin
245         for (int i = 0; i < 12; i++) begin
246             @(negedge wr_clk);

```

```

247         wr_en = 1;
248         data_in = test_data[i];
249         @(posedge wr_clk);
250         #1;
251         wr_en = 0;
252         #20; // Delay between writes
253     end
254 end
255
256 // Reader thread
257 begin
258     // Wait for first data to be available
259     #100;
260     for (int i = 0; i < 12; i++) begin
261         @(negedge rd_clk);
262         rd_en = 1;
263         @(posedge rd_clk);
264         #2;
265         if (data_out != test_data[i]) begin
266             $display("ERROR: Concurrent test data mismatch at %0d: expected=%h,
got=%h", i, test_data[i], data_out);
267             error_count++;
268         end
269         @(negedge rd_clk);
270         rd_en = 0;
271         #20; // Delay between reads
272     end
273 end
274 join
275
276 if (error_count == 0) begin
277     $display("PASS: Concurrent Read-Write Test");
278 end
279
280 // Test 7: Edge Case Test
281 $display("Test 7: Edge Case Test");
282
283 // Reset
284 rst_n = 0;
285 #200;
286 rst_n = 1;
287 #200;
288
289 // Write 15 entries (one before full)
290 for (int i = 0; i < 15; i++) begin
291     @(negedge wr_clk);
292     wr_en = 1;
293     data_in = test_data[i];
294     @(posedge wr_clk);
295     #1;
296     wr_en = 0;
297 end
298
299 #200; // Wait for synchronization
300
301 if (full != 1'b0 || almost_full != 1'b1) begin
302     $display("ERROR: Edge case - flags incorrect after 15 writes");
303     error_count++;
304 end
305
306 // Write 16th entry (full)
307 @(negedge wr_clk);

```

```

308     wr_en = 1;
309     data_in = test_data[15];
310     @(posedge wr_clk);
311     #1;
312     wr_en = 0;
313
314     // Wait a few more cycles for full flag to update
315     repeat (4) @(posedge wr_clk);
316     #200; // Wait for synchronization
317
318     if (full !== 1'b1) begin
319         $display("ERROR: Edge case - full flag not asserted");
320         error_count++;
321     end
322
323     // Read 13 entries to reach almost-empty threshold (3 remaining)
324     for (int i = 0; i < 13; i++) begin
325         @(negedge rd_clk);
326         rd_en = 1;
327         @(posedge rd_clk);
328         #2;
329         @(negedge rd_clk);
330         rd_en = 0;
331     end
332
333     #200; // Wait for synchronization
334
335     if (almost_empty !== 1'b1) begin
336         $display("ERROR: Edge case - almost empty flag not asserted");
337         error_count++;
338     end
339
340     // Read all remaining entries (empty)
341     for (int i = 0; i < 3; i++) begin
342         @(negedge rd_clk);
343         rd_en = 1;
344         @(posedge rd_clk);
345         #2;
346         @(negedge rd_clk);
347         rd_en = 0;
348     end
349
350     #200; // Wait for synchronization
351
352     if (empty !== 1'b1) begin
353         $display("ERROR: Edge case - empty flag not asserted");
354         error_count++;
355     end else begin
356         $display("PASS: Edge Case Test");
357     end
358
359     // Final result
360     if (error_count == 0) begin
361         $display("TestSucceeded: All tests passed!");
362     end else begin
363         $display("TestFailed: %0d errors found", error_count);
364     end
365
366     $finish;
367 end
368
369 // Monitor

```

```

370     initial begin
371         $monitor("Time=%0t: wr_en=%b, rd_en=%b, data_in=%h, data_out=%h, full=%b, empty=%b,
372             almost_full=%b, almost_empty=%b",
373             $time, wr_en, rd_en, data_in, data_out, full, empty, almost_full,
374             almost_empty);
375     end
endmodule

```

Listing 5: tb_async_fifo.sv

3 仿真结果

```

1  ### Simulation Result
2
3  Starting Async FIFO Test...
4  Test 1: Reset Test
5  Time=0: wr_en=0, rd_en=0, data_in=0000, data_out=0000, full=0, empty=1, almost_full=0,
6      almost_empty=1
7  PASS: Reset Test
8  Test 2: Basic Read-Write Test
9  Time=200: wr_en=1, rd_en=0, data_in=0000, data_out=0000, full=0, empty=1, almost_full=0,
10      almost_empty=1
11 Time=206: wr_en=0, rd_en=0, data_in=0000, data_out=0000, full=0, empty=1, almost_full=0,
12      almost_empty=1
13 Time=210: wr_en=1, rd_en=0, data_in=0001, data_out=0000, full=0, empty=1, almost_full=0,
14      almost_empty=1
15 Time=216: wr_en=0, rd_en=0, data_in=0001, data_out=0000, full=0, empty=1, almost_full=0,
16      almost_empty=1
17 Time=220: wr_en=1, rd_en=0, data_in=0002, data_out=0000, full=0, empty=1, almost_full=0,
18      almost_empty=1
19 Time=226: wr_en=0, rd_en=0, data_in=0002, data_out=0000, full=0, empty=1, almost_full=0,
20      almost_empty=1
21 Time=230: wr_en=1, rd_en=0, data_in=0003, data_out=0000, full=0, empty=1, almost_full=0,
22      almost_empty=1
23 Time=236: wr_en=0, rd_en=0, data_in=0003, data_out=0000, full=0, empty=1, almost_full=0,
24      almost_empty=1
25 Time=240: wr_en=1, rd_en=0, data_in=0004, data_out=0000, full=0, empty=1, almost_full=0,
26      almost_empty=1
27 Time=246: wr_en=0, rd_en=0, data_in=0004, data_out=0000, full=0, empty=1, almost_full=0,
28      almost_empty=1
29 Time=250: wr_en=1, rd_en=0, data_in=0005, data_out=0000, full=0, empty=0, almost_full=0,
30      almost_empty=1
31 Time=256: wr_en=0, rd_en=0, data_in=0005, data_out=0000, full=0, empty=0, almost_full=0,
32      almost_empty=1
33 Time=260: wr_en=1, rd_en=0, data_in=0006, data_out=0000, full=0, empty=0, almost_full=0,
34      almost_empty=1
35 Time=266: wr_en=0, rd_en=0, data_in=0006, data_out=0000, full=0, empty=0, almost_full=0,
36      almost_empty=1
37 Time=270: wr_en=1, rd_en=0, data_in=0007, data_out=0000, full=0, empty=0, almost_full=0,
38      almost_empty=1
39 Time=276: wr_en=0, rd_en=0, data_in=0007, data_out=0000, full=0, empty=0, almost_full=0,
40      almost_empty=1
41 Time=290: wr_en=0, rd_en=0, data_in=0007, data_out=0000, full=0, empty=0, almost_full=0,
42      almost_empty=0
43 Time=480: wr_en=0, rd_en=1, data_in=0007, data_out=0000, full=0, empty=0, almost_full=0,
44      almost_empty=0
45 Time=500: wr_en=0, rd_en=0, data_in=0007, data_out=0000, full=0, empty=0, almost_full=0,
46      almost_empty=0

```

```

27 Time=520: wr_en=0, rd_en=1, data_in=0007, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=0
28 Time=530: wr_en=0, rd_en=1, data_in=0007, data_out=0001, full=0, empty=0, almost_full=0,
    almost_empty=0
29 Time=540: wr_en=0, rd_en=0, data_in=0007, data_out=0001, full=0, empty=0, almost_full=0,
    almost_empty=0
30 Time=560: wr_en=0, rd_en=1, data_in=0007, data_out=0001, full=0, empty=0, almost_full=0,
    almost_empty=0
31 Time=570: wr_en=0, rd_en=1, data_in=0007, data_out=0002, full=0, empty=0, almost_full=0,
    almost_empty=0
32 Time=580: wr_en=0, rd_en=0, data_in=0007, data_out=0002, full=0, empty=0, almost_full=0,
    almost_empty=0
33 Time=600: wr_en=0, rd_en=1, data_in=0007, data_out=0002, full=0, empty=0, almost_full=0,
    almost_empty=0
34 Time=610: wr_en=0, rd_en=1, data_in=0007, data_out=0003, full=0, empty=0, almost_full=0,
    almost_empty=0
35 Time=620: wr_en=0, rd_en=0, data_in=0007, data_out=0003, full=0, empty=0, almost_full=0,
    almost_empty=0
36 Time=640: wr_en=0, rd_en=1, data_in=0007, data_out=0003, full=0, empty=0, almost_full=0,
    almost_empty=0
37 Time=650: wr_en=0, rd_en=1, data_in=0007, data_out=0004, full=0, empty=0, almost_full=0,
    almost_empty=0
38 Time=660: wr_en=0, rd_en=0, data_in=0007, data_out=0004, full=0, empty=0, almost_full=0,
    almost_empty=0
39 Time=670: wr_en=0, rd_en=0, data_in=0007, data_out=0004, full=0, empty=0, almost_full=0,
    almost_empty=1
40 Time=680: wr_en=0, rd_en=1, data_in=0007, data_out=0004, full=0, empty=0, almost_full=0,
    almost_empty=1
41 Time=690: wr_en=0, rd_en=1, data_in=0007, data_out=0005, full=0, empty=0, almost_full=0,
    almost_empty=1
42 Time=700: wr_en=0, rd_en=0, data_in=0007, data_out=0005, full=0, empty=0, almost_full=0,
    almost_empty=1
43 Time=720: wr_en=0, rd_en=1, data_in=0007, data_out=0005, full=0, empty=0, almost_full=0,
    almost_empty=1
44 Time=730: wr_en=0, rd_en=1, data_in=0007, data_out=0006, full=0, empty=0, almost_full=0,
    almost_empty=1
45 Time=740: wr_en=0, rd_en=0, data_in=0007, data_out=0006, full=0, empty=0, almost_full=0,
    almost_empty=1
46 Time=760: wr_en=0, rd_en=1, data_in=0007, data_out=0006, full=0, empty=0, almost_full=0,
    almost_empty=1
47 Time=770: wr_en=0, rd_en=1, data_in=0007, data_out=0007, full=0, empty=0, almost_full=0,
    almost_empty=1
48 PASS: Basic Read-Write Test
49 Test 3: Full Condition Test
50 Time=780: wr_en=0, rd_en=0, data_in=0007, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
51 Time=1180: wr_en=1, rd_en=0, data_in=0000, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
52 Time=1186: wr_en=0, rd_en=0, data_in=0000, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
53 Time=1190: wr_en=1, rd_en=0, data_in=0001, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
54 Time=1196: wr_en=0, rd_en=0, data_in=0001, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
55 Time=1200: wr_en=1, rd_en=0, data_in=0002, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
56 Time=1206: wr_en=0, rd_en=0, data_in=0002, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
57 Time=1210: wr_en=1, rd_en=0, data_in=0003, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
58 Time=1216: wr_en=0, rd_en=0, data_in=0003, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1

```

```

59 Time=1220: wr_en=1, rd_en=0, data_in=0004, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
60 Time=1226: wr_en=0, rd_en=0, data_in=0004, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
61 Time=1230: wr_en=1, rd_en=0, data_in=0005, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=1
62 Time=1236: wr_en=0, rd_en=0, data_in=0005, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=1
63 Time=1240: wr_en=1, rd_en=0, data_in=0006, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=1
64 Time=1246: wr_en=0, rd_en=0, data_in=0006, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=1
65 Time=1250: wr_en=1, rd_en=0, data_in=0007, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=1
66 Time=1256: wr_en=0, rd_en=0, data_in=0007, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=1
67 Time=1260: wr_en=1, rd_en=0, data_in=0008, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=1
68 Time=1266: wr_en=0, rd_en=0, data_in=0008, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=1
69 Time=1270: wr_en=1, rd_en=0, data_in=0009, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=0
70 Time=1276: wr_en=0, rd_en=0, data_in=0009, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=0
71 Time=1280: wr_en=1, rd_en=0, data_in=000a, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=0
72 Time=1285: wr_en=1, rd_en=0, data_in=000a, data_out=0000, full=0, empty=0, almost_full=1,
    almost_empty=0
73 Time=1286: wr_en=0, rd_en=0, data_in=000a, data_out=0000, full=0, empty=0, almost_full=1,
    almost_empty=0
74 Time=1290: wr_en=1, rd_en=0, data_in=000b, data_out=0000, full=0, empty=0, almost_full=1,
    almost_empty=0
75 Time=1296: wr_en=0, rd_en=0, data_in=000b, data_out=0000, full=0, empty=0, almost_full=1,
    almost_empty=0
76 Time=1300: wr_en=1, rd_en=0, data_in=000c, data_out=0000, full=0, empty=0, almost_full=1,
    almost_empty=0
77 Time=1306: wr_en=0, rd_en=0, data_in=000c, data_out=0000, full=0, empty=0, almost_full=1,
    almost_empty=0
78 Time=1310: wr_en=1, rd_en=0, data_in=000d, data_out=0000, full=0, empty=0, almost_full=1,
    almost_empty=0
79 Time=1316: wr_en=0, rd_en=0, data_in=000d, data_out=0000, full=0, empty=0, almost_full=1,
    almost_empty=0
80 Time=1320: wr_en=1, rd_en=0, data_in=000e, data_out=0000, full=0, empty=0, almost_full=1,
    almost_empty=0
81 Time=1326: wr_en=0, rd_en=0, data_in=000e, data_out=0000, full=0, empty=0, almost_full=1,
    almost_empty=0
82 Time=1330: wr_en=1, rd_en=0, data_in=000f, data_out=0000, full=0, empty=0, almost_full=1,
    almost_empty=0
83 Time=1335: wr_en=1, rd_en=0, data_in=000f, data_out=0000, full=1, empty=0, almost_full=1,
    almost_empty=0
84 Time=1336: wr_en=0, rd_en=0, data_in=000f, data_out=0000, full=1, empty=0, almost_full=1,
    almost_empty=0
85 Time=1580: wr_en=1, rd_en=0, data_in=ffff, data_out=0000, full=1, empty=0, almost_full=1,
    almost_empty=0
86 PASS: Full Condition Test
87 Test 4: Empty Condition Test
88 Time=1586: wr_en=0, rd_en=0, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
89 Time=2000: wr_en=0, rd_en=1, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
90 Time=2020: wr_en=0, rd_en=0, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1

```

```

91 Time=2040: wr_en=0, rd_en=1, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
92 Time=2060: wr_en=0, rd_en=0, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
93 Time=2080: wr_en=0, rd_en=1, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
94 Time=2100: wr_en=0, rd_en=0, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
95 Time=2120: wr_en=0, rd_en=1, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
96 Time=2140: wr_en=0, rd_en=0, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
97 Time=2160: wr_en=0, rd_en=1, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
98 Time=2180: wr_en=0, rd_en=0, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
99 Time=2200: wr_en=0, rd_en=1, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
100 Time=2220: wr_en=0, rd_en=0, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
101 Time=2240: wr_en=0, rd_en=1, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
102 Time=2260: wr_en=0, rd_en=0, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
103 Time=2280: wr_en=0, rd_en=1, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
104 Time=2300: wr_en=0, rd_en=0, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
105 Time=2320: wr_en=0, rd_en=1, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
106 Time=2340: wr_en=0, rd_en=0, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
107 Time=2360: wr_en=0, rd_en=1, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
108 Time=2380: wr_en=0, rd_en=0, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
109 Time=2400: wr_en=0, rd_en=1, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
110 Time=2420: wr_en=0, rd_en=0, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
111 Time=2440: wr_en=0, rd_en=1, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
112 Time=2460: wr_en=0, rd_en=0, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
113 Time=2480: wr_en=0, rd_en=1, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
114 Time=2500: wr_en=0, rd_en=0, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
115 Time=2520: wr_en=0, rd_en=1, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
116 Time=2540: wr_en=0, rd_en=0, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
117 Time=2560: wr_en=0, rd_en=1, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
118 Time=2580: wr_en=0, rd_en=0, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
119 Time=2600: wr_en=0, rd_en=1, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
120 Time=2620: wr_en=0, rd_en=0, data_in=ffff, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
121 PASS: Empty Condition Test
122 Test 5: Almost-Full/Almost-Empty Threshold Test

```

[illegible]

```
154 Time=3650: wr_en=0, rd_en=1, data_in=0009, data_out=0003, full=0, empty=0, almost_full=0,
    almost_empty=0
155 Time=3660: wr_en=0, rd_en=0, data_in=0009, data_out=0003, full=0, empty=0, almost_full=0,
    almost_empty=0
156 Time=3680: wr_en=0, rd_en=1, data_in=0009, data_out=0003, full=0, empty=0, almost_full=0,
    almost_empty=0
157 Time=3690: wr_en=0, rd_en=1, data_in=0009, data_out=0004, full=0, empty=0, almost_full=0,
    almost_empty=0
158 Time=3700: wr_en=0, rd_en=0, data_in=0009, data_out=0004, full=0, empty=0, almost_full=0,
    almost_empty=0
159 Time=3720: wr_en=0, rd_en=1, data_in=0009, data_out=0004, full=0, empty=0, almost_full=0,
    almost_empty=0
160 Time=3730: wr_en=0, rd_en=1, data_in=0009, data_out=0005, full=0, empty=0, almost_full=0,
    almost_empty=0
161 Time=3740: wr_en=0, rd_en=0, data_in=0009, data_out=0005, full=0, empty=0, almost_full=0,
    almost_empty=0
162 Time=3760: wr_en=0, rd_en=1, data_in=0009, data_out=0005, full=0, empty=0, almost_full=0,
    almost_empty=0
163 Time=3770: wr_en=0, rd_en=1, data_in=0009, data_out=0006, full=0, empty=0, almost_full=0,
    almost_empty=0
164 Time=3780: wr_en=0, rd_en=0, data_in=0009, data_out=0006, full=0, empty=0, almost_full=0,
    almost_empty=0
165 Time=3790: wr_en=0, rd_en=0, data_in=0009, data_out=0006, full=0, empty=0, almost_full=0,
    almost_empty=1
166 PASS: Almost-Full/Almost-Empty Threshold Test
167 Test 6: Concurrent Read-Write Test
168 Time=3980: wr_en=0, rd_en=0, data_in=0009, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
169 Time=4380: wr_en=1, rd_en=0, data_in=0000, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
170 Time=4386: wr_en=0, rd_en=0, data_in=0000, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
171 Time=4410: wr_en=1, rd_en=0, data_in=0001, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
172 Time=4416: wr_en=0, rd_en=0, data_in=0001, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
173 Time=4430: wr_en=0, rd_en=0, data_in=0001, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=1
174 Time=4440: wr_en=1, rd_en=0, data_in=0002, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=1
175 Time=4446: wr_en=0, rd_en=0, data_in=0002, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=1
176 Time=4470: wr_en=1, rd_en=0, data_in=0003, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=1
177 Time=4476: wr_en=0, rd_en=0, data_in=0003, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=1
178 Time=4480: wr_en=0, rd_en=1, data_in=0003, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=1
179 Time=4500: wr_en=1, rd_en=0, data_in=0004, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=1
180 Time=4506: wr_en=0, rd_en=0, data_in=0004, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=1
181 Time=4520: wr_en=0, rd_en=1, data_in=0004, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=1
182 Time=4530: wr_en=1, rd_en=1, data_in=0005, data_out=0001, full=0, empty=0, almost_full=0,
    almost_empty=1
183 Time=4536: wr_en=0, rd_en=1, data_in=0005, data_out=0001, full=0, empty=0, almost_full=0,
    almost_empty=1
184 Time=4540: wr_en=0, rd_en=0, data_in=0005, data_out=0001, full=0, empty=0, almost_full=0,
    almost_empty=1
185 Time=4560: wr_en=1, rd_en=1, data_in=0006, data_out=0001, full=0, empty=0, almost_full=0,
    almost_empty=1
```

```
186 Time=4566: wr_en=0, rd_en=1, data_in=0006, data_out=0001, full=0, empty=0, almost_full=0,
      almost_empty=1
187 Time=4570: wr_en=0, rd_en=1, data_in=0006, data_out=0002, full=0, empty=0, almost_full=0,
      almost_empty=1
188 Time=4580: wr_en=0, rd_en=0, data_in=0006, data_out=0002, full=0, empty=0, almost_full=0,
      almost_empty=1
189 Time=4590: wr_en=1, rd_en=0, data_in=0007, data_out=0002, full=0, empty=0, almost_full=0,
      almost_empty=1
190 Time=4596: wr_en=0, rd_en=0, data_in=0007, data_out=0002, full=0, empty=0, almost_full=0,
      almost_empty=1
191 Time=4600: wr_en=0, rd_en=1, data_in=0007, data_out=0002, full=0, empty=0, almost_full=0,
      almost_empty=1
192 Time=4610: wr_en=0, rd_en=1, data_in=0007, data_out=0003, full=0, empty=0, almost_full=0,
      almost_empty=0
193 Time=4620: wr_en=1, rd_en=0, data_in=0008, data_out=0003, full=0, empty=0, almost_full=0,
      almost_empty=0
194 Time=4626: wr_en=0, rd_en=0, data_in=0008, data_out=0003, full=0, empty=0, almost_full=0,
      almost_empty=0
195 Time=4630: wr_en=0, rd_en=0, data_in=0008, data_out=0003, full=0, empty=0, almost_full=0,
      almost_empty=1
196 Time=4640: wr_en=0, rd_en=1, data_in=0008, data_out=0003, full=0, empty=0, almost_full=0,
      almost_empty=1
197 Time=4650: wr_en=1, rd_en=1, data_in=0009, data_out=0004, full=0, empty=0, almost_full=0,
      almost_empty=0
198 Time=4656: wr_en=0, rd_en=1, data_in=0009, data_out=0004, full=0, empty=0, almost_full=0,
      almost_empty=0
199 Time=4660: wr_en=0, rd_en=0, data_in=0009, data_out=0004, full=0, empty=0, almost_full=0,
      almost_empty=0
200 Time=4680: wr_en=1, rd_en=1, data_in=000a, data_out=0004, full=0, empty=0, almost_full=0,
      almost_empty=0
201 Time=4686: wr_en=0, rd_en=1, data_in=000a, data_out=0004, full=0, empty=0, almost_full=0,
      almost_empty=0
202 Time=4690: wr_en=0, rd_en=1, data_in=000a, data_out=0005, full=0, empty=0, almost_full=0,
      almost_empty=0
203 Time=4700: wr_en=0, rd_en=0, data_in=000a, data_out=0005, full=0, empty=0, almost_full=0,
      almost_empty=0
204 Time=4710: wr_en=1, rd_en=0, data_in=000b, data_out=0005, full=0, empty=0, almost_full=0,
      almost_empty=0
205 Time=4716: wr_en=0, rd_en=0, data_in=000b, data_out=0005, full=0, empty=0, almost_full=0,
      almost_empty=0
206 Time=4720: wr_en=0, rd_en=1, data_in=000b, data_out=0005, full=0, empty=0, almost_full=0,
      almost_empty=0
207 Time=4730: wr_en=0, rd_en=1, data_in=000b, data_out=0006, full=0, empty=0, almost_full=0,
      almost_empty=0
208 Time=4740: wr_en=0, rd_en=0, data_in=000b, data_out=0006, full=0, empty=0, almost_full=0,
      almost_empty=0
209 Time=4760: wr_en=0, rd_en=1, data_in=000b, data_out=0006, full=0, empty=0, almost_full=0,
      almost_empty=0
210 Time=4770: wr_en=0, rd_en=1, data_in=000b, data_out=0007, full=0, empty=0, almost_full=0,
      almost_empty=0
211 Time=4780: wr_en=0, rd_en=0, data_in=000b, data_out=0007, full=0, empty=0, almost_full=0,
      almost_empty=0
212 Time=4800: wr_en=0, rd_en=1, data_in=000b, data_out=0007, full=0, empty=0, almost_full=0,
      almost_empty=0
213 Time=4810: wr_en=0, rd_en=1, data_in=000b, data_out=0008, full=0, empty=0, almost_full=0,
      almost_empty=0
214 Time=4820: wr_en=0, rd_en=0, data_in=000b, data_out=0008, full=0, empty=0, almost_full=0,
      almost_empty=0
215 Time=4830: wr_en=0, rd_en=0, data_in=000b, data_out=0008, full=0, empty=0, almost_full=0,
      almost_empty=1
216 Time=4840: wr_en=0, rd_en=1, data_in=000b, data_out=0008, full=0, empty=0, almost_full=0,
      almost_empty=1
```

```
217 Time=4850: wr_en=0, rd_en=1, data_in=000b, data_out=0009, full=0, empty=0, almost_full=0,
    almost_empty=1
218 Time=4860: wr_en=0, rd_en=0, data_in=000b, data_out=0009, full=0, empty=0, almost_full=0,
    almost_empty=1
219 Time=4880: wr_en=0, rd_en=1, data_in=000b, data_out=0009, full=0, empty=0, almost_full=0,
    almost_empty=1
220 Time=4890: wr_en=0, rd_en=1, data_in=000b, data_out=000a, full=0, empty=0, almost_full=0,
    almost_empty=1
221 Time=4900: wr_en=0, rd_en=0, data_in=000b, data_out=000a, full=0, empty=0, almost_full=0,
    almost_empty=1
222 Time=4920: wr_en=0, rd_en=1, data_in=000b, data_out=000a, full=0, empty=0, almost_full=0,
    almost_empty=1
223 Time=4930: wr_en=0, rd_en=1, data_in=000b, data_out=000b, full=0, empty=0, almost_full=0,
    almost_empty=1
224 Time=4940: wr_en=0, rd_en=0, data_in=000b, data_out=000b, full=0, empty=0, almost_full=0,
    almost_empty=1
225 Time=4950: wr_en=0, rd_en=0, data_in=000b, data_out=000b, full=0, empty=1, almost_full=0,
    almost_empty=1
226 PASS: Concurrent Read-Write Test
227 Test 7: Edge Case Test
228 Time=4960: wr_en=0, rd_en=0, data_in=000b, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
229 Time=5360: wr_en=1, rd_en=0, data_in=0000, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
230 Time=5366: wr_en=0, rd_en=0, data_in=0000, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
231 Time=5370: wr_en=1, rd_en=0, data_in=0001, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
232 Time=5376: wr_en=0, rd_en=0, data_in=0001, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
233 Time=5380: wr_en=1, rd_en=0, data_in=0002, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
234 Time=5386: wr_en=0, rd_en=0, data_in=0002, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
235 Time=5390: wr_en=1, rd_en=0, data_in=0003, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
236 Time=5396: wr_en=0, rd_en=0, data_in=0003, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
237 Time=5400: wr_en=1, rd_en=0, data_in=0004, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
238 Time=5406: wr_en=0, rd_en=0, data_in=0004, data_out=0000, full=0, empty=1, almost_full=0,
    almost_empty=1
239 Time=5410: wr_en=1, rd_en=0, data_in=0005, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=1
240 Time=5416: wr_en=0, rd_en=0, data_in=0005, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=1
241 Time=5420: wr_en=1, rd_en=0, data_in=0006, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=1
242 Time=5426: wr_en=0, rd_en=0, data_in=0006, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=1
243 Time=5430: wr_en=1, rd_en=0, data_in=0007, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=1
244 Time=5436: wr_en=0, rd_en=0, data_in=0007, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=1
245 Time=5440: wr_en=1, rd_en=0, data_in=0008, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=1
246 Time=5446: wr_en=0, rd_en=0, data_in=0008, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=1
247 Time=5450: wr_en=1, rd_en=0, data_in=0009, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=0
248 Time=5456: wr_en=0, rd_en=0, data_in=0009, data_out=0000, full=0, empty=0, almost_full=0,
    almost_empty=0
```

[illegible]

[illegible]

```
311 Time=6750: wr_en=0, rd_en=1, data_in=000f, data_out=000f, full=0, empty=0, almost_full=0,
    almost_empty=1
312 Time=6760: wr_en=0, rd_en=0, data_in=000f, data_out=000f, full=0, empty=0, almost_full=0,
    almost_empty=1
313 Time=6770: wr_en=0, rd_en=0, data_in=000f, data_out=000f, full=0, empty=1, almost_full=0,
    almost_empty=1
314 PASS: Edge Case Test
315 TestSucceeded: All tests passed!
```

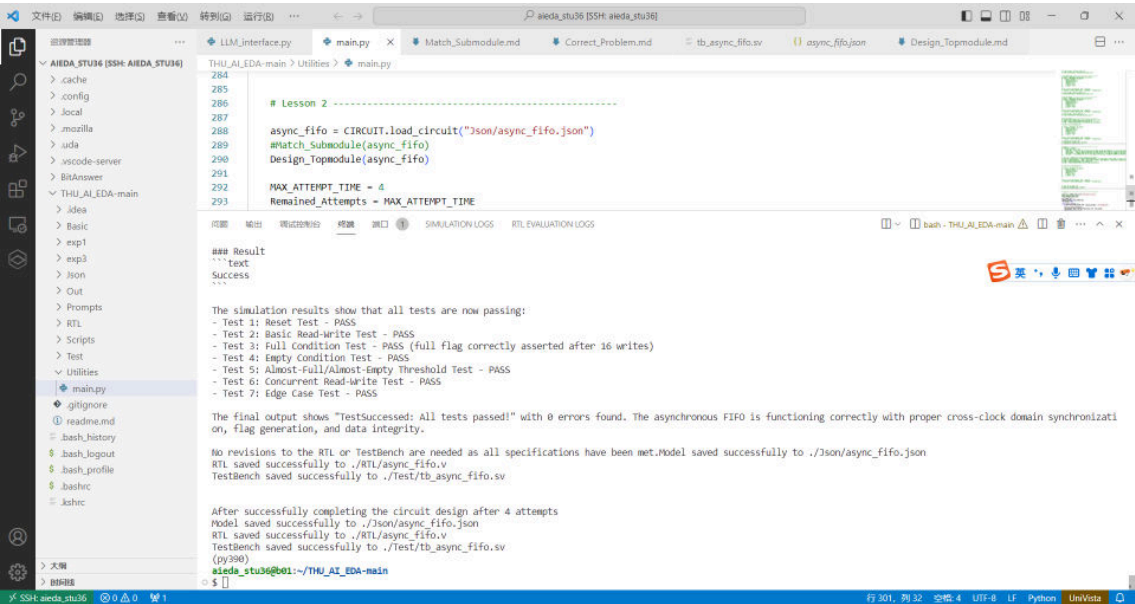


图 3: 通过

可以看到，最后的所有测试点均通过。我们通过搭建 Agent 系统，能让大模型在 4 步之内正确生成一个异步 FIFO。